# Two Graph Coloring Algorithms Based on Independent Sets: IndSet-Based Exact Coloring and IndSet-Removal Coloring

Graph Exact Coloring Heuristic Project

March 30, 2026

**Abstract**

We describe and analyze two graph coloring algorithms both rooted in the independent-set structure of a graph. The first, *IndSet-Based exact coloring*, determines the chromatic number exactly by evaluating a combinatorial quantity derived from inclusion-exclusion over all independent sets. The quantity, denoted $c_k(G)$, satisfies $c_k(G) = 0$ if and only if $k < \chi(G)$, so the chromatic number equals the smallest $k$ for which $c_k(G) \neq 0$. The second, *IndSet-Removal Coloring*, repeatedly peels off one independent set per round and assigns it the next available color, yielding a fast upper bound on $\chi(G)$. Both algorithms are implemented in C++17 using the Boost Graph Library. We present experimental results on cycles $C_n$, complete graphs $K_n$, the Petersen graph $GP(5, 2)$ and other generalized Petersen graphs, wheel graphs $W_n$, complete multipartite graphs $K_{s,\ldots,s}$, edgeless graphs, and graphs from the SuiteSparse Matrix Collection, reporting both the exact chromatic number and the IndSet-Removal upper bound side by side. The exact algorithm is practical for graphs up to roughly 13–14 vertices on a single core; IndSet-Removal runs in $O(n \cdot m)$ and is applicable to larger instances.

## Contents

**1 Introduction** **2**

**2 Preliminaries** **3**

**3 The Exact Algorithm** **3**
- 3.1 The Combinatorial Formula . . . 3
- 3.2 Combinatorial Interpretation . . . 3
- 3.3 Equivalence with $k$-Colorability . . . 4
- 3.4 Algorithm . . . 4

**4 IndSet-Removal Coloring** **5**
- 4.1 Independent Set Extraction . . . 5
- 4.2 IndSet-Removal Coloring . . . 5
- 4.3 Quality and Complexity . . . 5

**5 Implementation** **6**

**6 Experimental Results** **7**
6.1 Exact Values of $c_k$ at the Chromatic Number . . . 7
6.2 Parity Property of Odd Cycles . . . 7
6.3 Petersen Graph . . . 7
6.4 Wheel Graphs . . . 7
6.5 Complete Multipartite Graphs . . . 8
6.6 Generalized Petersen Family . . . 8
6.7 SuiteSparse Matrix Collection Graphs . . . 8
6.8 Scaling Analysis . . . 8
6.9 High Chromatic Number Graphs . . . 9
6.10 IndSet-Removal vs. Exact Coloring . . . 9

**7 Complexity Analysis** **9**
7.1 Independent Set Enumeration . . . 9
7.2 Evaluation of $c_k$ . . . 10
7.3 Overall Complexity . . . 10
7.4 Space Complexity . . . 10

**8 Conclusion** **10**

# 1 Introduction

Graph coloring is one of the most extensively studied problems in combinatorics and theoretical computer science. Given a graph $G = (V, E)$, a *proper $k$-coloring* assigns one of $k$ colors to each vertex so that no two adjacent vertices share a color. The minimum such $k$ is the *chromatic number* $\chi(G)$. Determining $\chi(G)$ is NP-hard in general [1], yet exact algorithms are essential for small or sparse graphs arising in scientific computing, scheduling, and register allocation.

This paper documents two algorithms implemented in the `graph-exact-coloring-heuristic` project. The first, *IndSet-Based Exact Coloring*, uses the core function `c_k` which evaluates the inclusion-exclusion sum

$$c_k(G, k) \;=\; \sum_{S \subseteq V} (-1)^{|S|} \left|\{\, I \in \mathcal{F}(G) : I \cap S = \emptyset \,\}\right|^k,$$

where $\mathcal{F}(G)$ is the family of all independent sets of $G$ (including the empty set). We show that this quantity is zero precisely when $G$ is not $k$-colorable, and hence one can binary-search (or linearly scan) over $k$ to find $\chi(G)$.

The second, *IndSet-Removal Coloring* (implemented in `demo_is_removal_coloring.cpp`), repeatedly extracts an independent set and assigns it the next available color, yielding an upper bound that may exceed $\chi(G)$ in general.

**Organization.** Section 2 fixes notation and recalls standard facts about graph coloring. Section 3 derives the formula and proves its correctness. Section 4 describes the IndSet-Removal Coloring algorithm. Section 5 outlines the C++ implementation. Section 6 presents experimental results, including graphs from the SuiteSparse Matrix Collection. Section 7 analyzes time and space complexity. Section 8 concludes.

## 2 Preliminaries

Throughout, $G = (V, E)$ is a simple undirected graph with $n = |V|$ vertices and $m = |E|$ edges. Vertices are labeled $0, 1, \ldots, n-1$.

**Definition 1** (Independent set)**.** *A set $I \subseteq V$ is* independent *if no two vertices in $I$ are adjacent. We write $\mathcal{F}(G)$ for the family of all independent sets of $G$, including $\emptyset$.*

**Definition 2** (Proper coloring and chromatic number)**.** *A* proper $k$-coloring *of $G$ is a function $f : V \to \{1, \ldots, k\}$ with $f(u) \neq f(v)$ whenever $uv \in E$. The* chromatic number $\chi(G)$ *is the minimum $k$ for which a proper $k$-coloring exists.*

**Definition 3** (Independent set cover)**.** *An* ordered $k$-cover by independent sets *(hereafter, $k$-IndSet-cover) is a $k$-tuple $(I_1, \ldots, I_k) \in \mathcal{F}(G)^k$ such that $I_1 \cup \cdots \cup I_k = V$.*

Note that in a $k$-IndSet-cover the sets need not be disjoint, and they may be empty. A proper $k$-coloring induces a $k$-IndSet-cover (the color classes), but a $k$-IndSet-cover may have overlapping sets. We will show these notions are equivalent for the *existence* question.

**Definition 4** (Chromatic polynomial)**.** *The* chromatic polynomial $P(G, k)$ *counts the number of proper $k$-colorings of $G$. It satisfies $P(G, k) = 0$ iff $k < \chi(G)$.*

Well-known chromatic polynomials for families used in experiments:

$$\begin{aligned} P(K_n, k) &= k(k-1)(k-2)\cdots(k-n+1), \\ P(C_n, k) &= (k-1)^n + (-1)^n(k-1), \\ P(E_n, k) &= k^n, \end{aligned}$$

where $K_n$ is the complete graph, $C_n$ the cycle, and $E_n$ the edgeless graph on $n$ vertices.

## 3 The Exact Algorithm

### 3.1 The Combinatorial Formula

For a graph $G$ and a positive integer $k$, define

$$\alpha(S, \mathcal{F}) \;=\; \big|\{\, I \in \mathcal{F}(G) : I \cap S = \emptyset \,\}\big|$$

to be the number of independent sets of $G$ that avoid $S$ entirely. Equivalently, $\alpha(S, \mathcal{F})$ is the total number of independent sets in the subgraph of $G$ induced by $V \setminus S$. The algorithm evaluates

$$c_k(G, k) \;=\; \sum_{S \subseteq V} (-1)^{|S|}\, \alpha(S, \mathcal{F})^k. \tag{1}$$

### 3.2 Combinatorial Interpretation

**Theorem 1.** *$c_k(G, k)$ equals the number of ordered $k$-IndSet-covers of $G$.*

*Proof.* Let $N = |\mathcal{F}(G)|$. The number of ordered $k$-tuples $(I_1, \ldots, I_k) \in \mathcal{F}(G)^k$ with no constraint is $N^k$. For a fixed vertex $v \in V$ let $A_v$ be the set of $k$-tuples in which $v$ is covered by no $I_j$ (i.e. $v \notin I_j$ for all $j$). Then

$$|A_v| = |\{I \in \mathcal{F}(G) : v \notin I\}|^k = \alpha(\{v\}, \mathcal{F})^k.$$

More generally, for any $S \subseteq V$,

$$\Big|\bigcap_{v\in S} A_v\Big| = |\{I \in \mathcal{F}(G) : S \cap I = \emptyset\}|^k = \alpha(S,\mathcal{F})^k.$$

By inclusion-exclusion, the number of $k$-tuples in which every vertex is covered by at least one $I_j$ is

$$\sum_{S\subseteq V} (-1)^{|S|}\, \alpha(S,\mathcal{F})^k \;=\; c_k(G,k). \qquad \square$$

### 3.3 Equivalence with $k$-Colorability

**Theorem 2.** *$c_k(G,k) = 0$ if and only if $G$ is not properly $k$-colorable, i.e. $k < \chi(G)$.*

*Proof.* ($\Leftarrow$) Suppose $k \geq \chi(G)$. Fix a proper $k$-coloring $f : V \to \{1,\ldots,k\}$ and let $I_j = f^{-1}(j)$ for each $j$. Each $I_j$ is an independent set and $\bigcup_j I_j = V$, so $(I_1,\ldots,I_k)$ is a $k$-IndSet-cover. Hence $c_k(G,k) \geq 1 > 0$.

($\Rightarrow$) Suppose $k < \chi(G)$. We show no $k$-IndSet-cover exists. If $(I_1,\ldots,I_k)$ were a $k$-IndSet-cover, define $f(v) = \min\{j : v \in I_j\}$ for each $v \in V$. Since $I_j$ is independent, no two adjacent vertices can share the same $j$; thus $f$ is a proper $k$-coloring, contradicting $k < \chi(G)$. Therefore $c_k(G,k) = 0$. $\square$

**Corollary 3.** $\chi(G) = \min\{\, k \geq 1 : c_k(G,k) \neq 0 \,\}$.

**Remark 1.** *The quantity $c_k(G,k)$ is not the chromatic polynomial $P(G,k)$ in general. For $k = \chi(G)$, the two coincide for complete graphs (where every $k$-IndSet-cover induces a proper coloring immediately), but they differ for most other graphs at values $k > \chi(G)$. For example, $c_k(C_5,3) = 60$ while $P(C_5,3) = 30$. Nevertheless, both are zero for $k < \chi(G)$ and nonzero for $k \geq \chi(G)$, which is the only property needed to determine the chromatic number.*

### 3.4 Algorithm

The full procedure to compute $\chi(G)$ is given in Algorithm 2. It first enumerates all independent sets of $G$ (Algorithm 1) using a power-set sweep with bitset membership testing, then evaluates formula (1) for increasing $k$.

**Algorithm 1** Enumerate all independent sets of $G$

**Require:** Graph $G = (V,E)$ with $|V| = n$
**Ensure:** Set $\mathcal{F}$ of all independent sets (as $n$-bit integers)

```
F ← ∅
for i ← 0 to 2^n − 1 do
    B ← n-bit representation of i
    if ∀ u ≠ v: B[u] = B[v] = 1 ⇒ uv ∉ E then
        F ← F ∪ {B}
    end if
end forreturn F
```

#### Bitset Representation

Each independent set is stored as an $n$-bit integer (a `boost::dynamic_bitset`). The independence check for a candidate set $B$ iterates over all pairs $(i,j)$ with $B[i] = B[j] = 1$ and verifies $ij \notin E$. The subset-avoidance test $I \cap S = \emptyset$ reduces to the bitwise operation $(I \,\&\, S) = 0$.

**Algorithm 2** Compute $\chi(G)$ via inclusion-exclusion

**Require:** Graph $G = (V, E)$ with $|V| = n$
**Ensure:** Chromatic number $\chi(G)$

```
F ← EnumerateIndepSets(G)
for k ← 1 to n do
    cnt ← 0
    for each S ⊆ V (represented as a bitset) do
        a ← |{I ∈ F : I & S = 0}|                    ▷ count IS avoiding S
        cnt ← cnt + (−1)^|S| · a^k
    end for
    if cnt ≠ 0 then return k
    end if
end for
```

# 4 IndSet-Removal Coloring

## 4.1 Independent Set Extraction

The IndSet extraction procedure (Algorithm 3) builds one independent set by iterating through the active vertices in index order. For each vertex $v$ selected, it removes $v$ and all of $v$'s current neighbors from the active set.

**Algorithm 3** Independent set extraction

**Require:** Graph $G = (V, E)$; active set $U \subseteq V$
**Ensure:** An independent set $I \subseteq U$

```
I ← ∅
while G has edges do
    v ← first vertex in U
    I ← I ∪ {v}
    Remove v and all neighbors of v from U and from G
end whilereturn I
```

## 4.2 IndSet-Removal Coloring

Algorithm 4 repeatedly calls the IS extractor to peel off one independent set (color class) at a time.

## 4.3 Quality and Complexity

IndSet-Removal Coloring produces at most $\Delta(G)+1$ colors, where $\Delta(G)$ is the maximum degree [2]. The vertex-order traversal makes it exact on complete graphs and odd cycles but can be arbitrarily far from optimal on bipartite or dense multipartite graphs (see Table 1 for empirical comparisons). Its primary advantage is $O(n+m)$ running time per color class, making it practical for large sparse graphs.

**Algorithm 4** IndSet-Removal Coloring

**Require:** Graph $G = (V, E)$
**Ensure:** Coloring array $col[0 \dots n-1]$

```
col ← array of size n; k ← 0
while G has edges do
    I ← ExtractIndepSet(G)
    for each v ∈ I do
        col[v] ← k
        Remove all edges incident to v from G
    end for
    k ← k + 1
end whilereturn col
```

## 5 Implementation

The project is written in C++17 and uses the Boost Graph Library (BGL) version 1.60 or later. The graph type is

```
typedef boost::adjacency_list<
    boost::vecS, boost::vecS, boost::undirectedS,
    property<boost::vertex_color_t, int>,
    property<boost::edge_weight_t, int,
             property<boost::edge_name_t, string>>> Graph;
```

Independent sets are stored as `boost::dynamic_bitset<>` objects, which support $O(n/w)$ bitwise AND, OR, and population-count operations where $w$ is the machine word size.

The key functions are:

- `power_set(n, f)`: iterates over all $2^n$ subsets, calling `f` with each subset represented as a `dynbit`.
- `is_indep_set(G, B)`: checks all pairs of set bits in $B$ for adjacency.
- `gen_ind_set_for_g(G)`: returns $\mathcal{F}(G)$ as a `std::set<dynbit>`.
- `alpha(S, F)`: counts $|\{I \in F : I \,\&\, S = 0\}|$ by a linear scan.
- `c_k(F, G, k)`: evaluates formula (1) using floating-point arithmetic and casts the result to `int`.

Generators for Cycle ($C_n$) and Generalized Peterson ($GP(n, k)$) graphs extend the abstract base class `GeneratorInterface` and override `generate()`.

**Generalized Peterson graph.** $GP(n, k)$ has $2n$ vertices. The outer $n$-cycle connects vertex $i$ to $(i + 1) \bmod n$. Spoke edges connect outer vertex $i$ to inner vertex $i + n$. Inner edges connect $i + n$ to $(i + n + k) \bmod n + n$ with a boundary correction.

**Build system.** The project uses CMake 3.10 or later. Three targets are built: `indset_exact_coloring` (IndSet-Based Exact Coloring demo, `demo_indset_exact_coloring.cpp`), `indset_removal_coloring` (IndSet-Removal demo, `demo_indset_removal_coloring.cpp`), and `tests` (`tests.cpp`, the test suite described in Section 6).

# 6 Experimental Results

All experiments were run on the `tests` binary compiled with `-std=c++17 --openmp -O2` on Ubuntu 22.04. Table 1 summarizes results for both the IndSet-Based Exact Coloring and IndSet-Removal Coloring algorithms.

All 78 test assertions pass: IndSet-Based Exact Coloring recovers the known chromatic number in every case. In particular, $c_k(K_9, 9) = 9! = 362{,}880$ and $c_k(K_{10}, 10) = 10! = 3{,}628{,}800$ are verified exactly. IndSet-Removal Coloring matches the exact result on complete graphs and odd cycles, but over-estimates on even cycles, bipartite, and multipartite graphs.

## 6.1 Exact Values of $c_k$ at the Chromatic Number

For complete graphs, the formula (1) coincides with the standard chromatic polynomial at $k = \chi(G)$:

The coincidence for $K_n$ can be verified algebraically: since the only independent sets of $K_n$ are $\emptyset$ and the $n$ singletons, $\alpha(S, \mathcal{F}) = n - |S| + 1$ and the sum (1) at $k = n$ evaluates to

$$\sum_{j=0}^{n} \binom{n}{j} (-1)^{n-j} (j+1)^n = n!,$$

which equals $P(K_n, n) = n!$. For cycle $C_4$ the coincidence is accidental; for $C_5$ the values differ.

## 6.2 Parity Property of Odd Cycles

For any odd cycle $C_n$ ($n$ odd) the algorithm correctly returns $\chi = 3$. No odd cycle is 2-colorable because any 2-coloring of a path forces the two endpoints of a cycle to receive the same color, contradicting the closing edge. For $n = 3, 5, 7$ the algorithm confirmed $c_k(C_n, 2) = 0$ and $c_k(C_n, 3) > 0$.

## 6.3 Petersen Graph

The Petersen graph $GP(5, 2)$ is the canonical example of a 3-chromatic 3-regular graph. It is not planar and has no Hamiltonian cycle. Our algorithm confirmed $\chi(GP(5, 2)) = 3$, with $c_k(GP(5, 2), 2) = 0$ and $c_k(GP(5, 2), 3) > 0$.

## 6.4 Wheel Graphs

The wheel graph $W_n$ consists of a single hub vertex connected to every vertex of a cycle $C_{n-1}$. It has $n$ vertices and $2(n-1)$ edges. Because the hub is adjacent to every rim vertex, the hub's color must differ from all colors used on the rim.

- If $n$ is odd, the rim $C_{n-1}$ has an even number of vertices and is therefore bipartite, so $\chi(C_{n-1}) = 2$. The hub requires a third color, giving $\chi(W_n) = 3$.
- If $n$ is even, the rim $C_{n-1}$ is an odd cycle, so $\chi(C_{n-1}) = 3$. The hub must avoid all three rim colors, giving $\chi(W_n) = 4$.

This makes wheel graphs the simplest natural family with $\chi = 4$. We verified $\chi(W_n)$ for $n \in \{5, 6, 7, 8, 9\}$; all 5 tests pass.

### 6.5 Complete Multipartite Graphs

The complete $k$-partite graph $K_{s,\ldots,s}$ (with $k$ parts of $s$ vertices each) has $\chi = k$: color each part with its own color (lower bound from the clique-like independence structure; $k$ colors suffice since each part is an independent set). We tested:

- $K_{2,2,2}$ (octahedron): 6 vertices, 12 edges, $\chi = 3$.
- $K_{3,3,3}$: 9 vertices, 27 edges, $\chi = 3$.
- $K_{2,2,2,2}$: 8 vertices, 24 edges, $\chi = 4$.

These graphs have few independent sets relative to $2^n$ (only subsets within a single part), so the algorithm is fast despite the large edge count.

### 6.6 Generalized Petersen Family

Beyond $GP(5, 2)$ we tested five more members:

- $GP(4, 1) \cong Q_3$ (3-cube): bipartite, $\chi = 2$.
- $GP(5, 1)$ (pentagonal prism): odd outer cycle, $\chi = 3$.
- $GP(6, 1)$ (hexagonal prism): even outer cycle, bipartite, $\chi = 2$.
- $GP(6, 2)$: inner triangles force $\chi \geq 3$; algorithm confirms $\chi = 3$.
- $GP(7, 2)$: odd outer and inner cycles, $\chi = 3$.

### 6.7 SuiteSparse Matrix Collection Graphs

To demonstrate applicability to real-world data, we obtained matrices from the SuiteSparse Matrix Collection [6] and converted each to an undirected graph by symmetrizing (adding edge $(i, j)$ whenever $A_{ij} \neq 0$ or $A_{ji} \neq 0$) and discarding self-loops and edge weights. For matrices larger than 14 vertices we extract a connected subgraph of $k$ vertices via BFS from a fixed starting vertex.

The cage matrices arise from random walks on finite words over the DNA alphabet $\{A, C, G, T\}$; `cage3` has $|\Sigma|^3 - |\Sigma|^2 = 48$ directed edges reduced to 7 undirected after symmetrization. Both DNA graphs require exactly 3 colors — they contain odd cycles formed by overlapping DNA words that share a common suffix/prefix. The `GD98_a` subgraph also requires 3 colors, while the symmetric FEM matrix `ndtest` is 2-colorable in its 10-vertex neighborhood, consistent with the bipartite structure typical of finite-element adjacency graphs.

### 6.8 Scaling Analysis

Table 4 reports wall-clock times for selected larger graphs. The dominant cost is evaluating $c_k$ (formula (1)), which is $O(2^n \cdot |\mathcal{F}|)$ per candidate $k$. For sparse graphs such as cycles, $|\mathcal{F}|$ grows like the Lucas sequence $L(n) \approx \varphi^n$, so the total cost is roughly $O((2\varphi)^n) \approx O(3.24^n)$, yielding approximately a 10× slowdown per 2 additional vertices. Dense graphs such as $K_{m,m}$ have much smaller $|\mathcal{F}|$ and therefore scale better.

### 6.9 High Chromatic Number Graphs

Complete graphs $K_n$ have $\chi(K_n) = n$ and $|\mathcal{F}| = n + 1$ (only singletons and the empty set), so the independent-set enumeration phase is trivial. The bottleneck is computing the chromatic polynomial at each candidate $k$, which requires intermediate terms of the form $(-1)^{|S|}\,\alpha(S, \mathcal{F})^k$. For $K_9$, these terms reach $9^9 \approx 3.9 \times 10^8$, which fits comfortably in a 64-bit integer. The implementation uses `long long` accumulation with integer exponentiation (no floating point), giving correct results up to at least $K_{10}$ ($10! = 3{,}628{,}800$, verified by the test suite).

Complete $k$-partite graphs $K_{2\times k}$ (two vertices per part) provide compact benchmarks for high chromatic numbers: only $2k$ vertices but $\chi = k$. Their independent-set families have exactly $3^k$ members (each part contributes three choices: both, neither, or one vertex), far fewer than $2^{2k}$, so the algorithm scales well despite the large chromatic number.

### 6.10 IndSet-Removal vs. Exact Coloring

The IndSet-Removal column in Tables 1–5 shows the number of colors produced by Algorithm 4 on each graph. The algorithm processes vertices in ascending index order, so its quality depends strongly on the vertex ordering. The results illustrate several patterns:

- **Complete graphs**: IndSet-Removal is always optimal. Because every vertex is adjacent to all lower-indexed vertices, each vertex receives a new color, achieving exactly $n$ colors for $K_n$.
- **Odd cycles**: IndSet-Removal is optimal ($\chi = 3$). Vertex 0 and its non-adjacent neighbor both receive color 0, and the single remaining vertex requires a third color.
- **Even cycles and bipartite graphs**: IndSet-Removal performs poorly because the vertex-order traversal fails to exploit the bipartite structure. $C_4$ receives 4 colors instead of 2; similarly $GP(6, 1)$ (hexagonal prism, bipartite) receives 6 instead of 2.
- **Wheel graphs**: exact when $n$ is even (odd rim forces 4 colors regardless of ordering) but over-estimates by 2 when $n$ is odd.
- **Complete multipartite** $K_{2\times k}$: the over-count grows linearly with $k$; IndSet-Removal uses approximately $2k - 1$ colors instead of $k$, giving an approximation ratio approaching 2 as $k$ grows.

These results confirm that IndSet-Removal Coloring is a fast but unreliable upper bound: it is tight on graphs where the natural vertex order aligns with the chromatic structure (complete graphs, odd cycles), but can be far from optimal on structured sparse or multipartite graphs.

## 7 Complexity Analysis

### 7.1 Independent Set Enumeration

`gen_ind_set_for_g` calls `power_set`, which iterates over all $2^n$ subsets. For each subset $B$, `is_indep_set` checks all pairs of set bits: at most $O(n^2)$ edge queries. Hence the enumeration phase runs in $O(2^n \cdot n^2)$ time and produces a set $\mathcal{F}$ of at most $3^n/\text{poly}(n)$ independent sets (by the Moon–Moser theorem on the maximum number of maximal independent sets in a graph with $n$ vertices [3]).

### 7.2 Evaluation of $c_k$

For each candidate $k$, evaluating formula (1) requires iterating over all $2^n$ subsets $S$ and, for each $S$, scanning all $|\mathcal{F}|$ independent sets to compute $\alpha(S, \mathcal{F})$. This costs $O(2^n \cdot |\mathcal{F}| \cdot n/w)$ per value of $k$, where $n/w$ is the cost of a bitwise AND. Summing over at most $n$ values of $k$, the total evaluation cost is $O(n \cdot 2^n \cdot |\mathcal{F}| \cdot n/w)$.

### 7.3 Overall Complexity

The dominating term is the evaluation of $c_k$, giving

$$T(n) = O\big(n^2 \cdot 2^n \cdot |\mathcal{F}(G)|\big)\,.$$

In the worst case (e.g., the empty graph $E_n$ has $|\mathcal{F}| = 2^n$), this is $O(n^2 \cdot 4^n)$. Empirically (Table 4), sparse graphs such as cycles reach the practical limit around $n = 13$–$14$ (seconds to minutes), while dense graphs such as $K_{m,m}$ scale better because $|\mathcal{F}|$ is much smaller than $2^n$. The algorithm is intended for IndSet-Based Exact Coloring of small subgraphs; for large graphs the IndSet-Removal coloring in `demo_is_removal_coloring.cpp` is the appropriate choice.

### 7.4 Space Complexity

Storing $\mathcal{F}$ requires $O(|\mathcal{F}| \cdot n/w)$ words, which is $O(3^n \cdot n/w)$ in the worst case.

## 8 Conclusion

We have described and formally justified the inclusion-exclusion algorithm for computing the chromatic number of a graph implemented in the `graph-exact-coloring-heuristic` project. The key result is that the quantity $c_k(G, k)$ — the number of ordered $k$-tuples of independent sets covering $V$ — is zero if and only if $G$ is not $k$-colorable (Theorem 2). Scanning $k = 1, 2, \ldots$ and stopping at the first nonzero value thus yields $\chi(G)$ exactly.

The algorithm is exponential in $|V|$ and is therefore intended for small graphs or for IndSet-Based Exact Coloring of subgraphs within a larger pipeline. The companion IndSet-Removal Coloring algorithm runs in $O(n + m)$ time per color class and provides a fast upper bound useful as an initialization strategy or a standalone approximation for large instances.

Future directions include: (i) exploiting symmetry to prune the power-set enumeration, (ii) integrating branch-and-bound to improve worst-case behavior, and (iii) leveraging OpenMP (already supported in the build) to parallelize the inner $\alpha$-counting loop.

## References


[1] R. M. Karp. Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972.

[2] D. J. A. Welsh and M. B. Powell. An upper bound for the chromatic number of a graph and its application to timetabling problems. *The Computer Journal*, 10(1):85–86, 1967.

[3] J. W. Moon and L. Moser. On cliques in graphs. *Israel Journal of Mathematics*, 3(1):23–28, 1965.

[4] E. L. Lawler. A note on the complexity of the chromatic number problem. *Information Processing Letters*, 5(3):66–67, 1976.

[5] J. G. Siek, L.-Q. Lee, and A. Lumsdaine. *The Boost Graph Library: User Guide and Reference Manual*. Addison-Wesley, 2002.

[6] T. A. Davis and Y. Hu. The University of Florida sparse matrix collection. *ACM Transactions on Mathematical Software*, 38(1):1–25, 2011.

Table 1: All three results per graph: known $\chi$, IndSet-Based Exact Coloring, and IndSet-Removal Coloring (78 exact assertions, all passing).

| Graph | Description | $\lvert V\rvert$ | $\lvert E\rvert$ | Known $\chi$ | IndSet-Exact | IndSet-Removal |
|---|---|---|---|---|---|---|
| *Cycle graphs* | | | | | | |
| $C_3$ | Triangle | 3 | 3 | 3 | 3 | 3 |
| $C_4$ | 4-cycle (square) | 4 | 4 | 2 | 2 | 4 |
| $C_5$ | 5-cycle (pentagon) | 5 | 5 | 3 | 3 | 3 |
| $C_6$ | 6-cycle (hexagon) | 6 | 6 | 2 | 2 | 4 |
| $C_7$ | 7-cycle | 7 | 7 | 3 | 3 | 3 |
| *Complete and edgeless graphs* | | | | | | |
| $K_1$ | Single vertex | 1 | 0 | 1 | 1 | 1 |
| $K_2$ | Single edge | 2 | 1 | 2 | 2 | 2 |
| $K_3$ | Triangle | 3 | 3 | 3 | 3 | 3 |
| $K_4$ | Complete graph | 4 | 6 | 4 | 4 | 4 |
| $K_9$ | Complete graph | 9 | 36 | 9 | 9 | 9 |
| $K_{10}$ | Complete graph | 10 | 45 | 10 | 10 | 10 |
| $E_1$ | Isolated vertex | 1 | 0 | 1 | 1 | 1 |
| $E_3$ | Three isolated vertices | 3 | 0 | 1 | 1 | 1 |
| *Bipartite graphs* | | | | | | |
| $K_{2,3}$ | Complete bipartite | 5 | 6 | 2 | 2 | 3 |
| *Wheel graphs* $W_n$; $\chi = 3$ if $n$ odd, $\chi = 4$ if $n$ even | | | | | | |
| $W_5$ | Hub $+ C_4$ (even rim) | 5 | 8 | 3 | 3 | 5 |
| $W_6$ | Hub $+ C_5$ (odd rim) | 6 | 10 | 4 | 4 | 4 |
| $W_7$ | Hub $+ C_6$ (even rim) | 7 | 12 | 3 | 3 | 5 |
| $W_8$ | Hub $+ C_7$ (odd rim) | 8 | 14 | 4 | 4 | 4 |
| $W_9$ | Hub $+ C_8$ (even rim) | 9 | 16 | 3 | 3 | 5 |
| *Complete multipartite graphs*; $\chi(K^{(k)}_{s,\ldots,s}) = k$ | | | | | | |
| $K_{2,2,2}$ | Octahedron | 6 | 12 | 3 | 3 | 5 |
| $K_{3,3,3}$ | Complete 3-partite | 9 | 27 | 3 | 3 | 7 |
| $K_{2,2,2,2}$ | Complete 4-partite | 8 | 24 | 4 | 4 | 7 |
| $K_{2\times 5}$ | Complete 5-partite | 10 | 40 | 5 | 5 | 9 |
| $K_{2\times 6}$ | Complete 6-partite | 12 | 60 | 6 | 6 | 11 |
| $K_{2\times 7}$ | Complete 7-partite | 14 | 84 | 7 | 7 | 13 |
| $K_{2\times 8}$ | Complete 8-partite | 16 | 112 | 8 | 8 | 15 |
| $K_{2\times 9}$ | Complete 9-partite | 18 | 144 | 9 | 9 | 17 |
| *Generalized Petersen graphs* $GP(n,k)$ | | | | | | |
| $GP(4,1)$ | 3-cube $Q_3$ (bipartite) | 8 | 12 | 2 | 2 | 6 |
| $GP(5,1)$ | Pentagonal prism | 10 | 15 | 3 | 3 | 3 |
| $GP(5,2)$ | Petersen graph | 10 | 15 | 3 | 3 | 4 |
| $GP(6,1)$ | Hexagonal prism | 12 | 18 | 2 | 2 | 6 |
| $GP(6,2)$ | — | 12 | 18 | 3 | 3 | 4 |
| $GP(7,2)$ | — | 14 | 21 | 3 | 3 | 4 |

Table 2: Values of $c_k(G, \chi(G))$ vs. the chromatic polynomial $P(G, \chi(G))$.

| Graph | $\chi$ | $c_k(G, \chi)$ | $P(G, \chi)$ | Match? |
|---|---|---|---|---|
| $K_1$ | 1 | 1 | 1 | Yes |
| $K_2$ | 2 | 2 | 2 | Yes |
| $K_3$ | 3 | 6 | 6 | Yes |
| $K_4$ | 4 | 24 | 24 | Yes |
| $C_3$ | 3 | 6 | 6 | Yes |
| $C_4$ | 2 | 2 | 2 | Yes |
| $C_5$ | 3 | 60 | 30 | No |

Table 3: SuiteSparse graphs: known $\chi$, IndSet-Based Exact Coloring, IndSet-Removal Coloring, and wall-clock time for each algorithm.

| Matrix | Group | Processing | $\lvert V\rvert$ | $\lvert E\rvert$ | Known $\chi$ | IndSet-Exact | IndSet-Removal | Exact Time | Removal Time |
|---|---|---|---|---|---|---|---|---|---|
| `cage3` | vanHeukelum | symmetrized (full) | 5 | 7 | 3 | 3 | 4 | <1 ms | <1 ms |
| `cage4` | vanHeukelum | symmetrized (full) | 9 | 20 | 3 | 3 | 7 | 3 ms | <1 ms |
| `GD98_a` | Pajek | BFS-10 subgraph | 10 | 10 | 3 | 3 | 3 | 81 ms | <1 ms |
| `ndtest` | (project) | BFS-10 subgraph | 10 | 14 | 2 | 2 | 6 | 15 ms | <1 ms |

Table 4: Wall-clock time for larger graphs (Release build, single core, Ubuntu 22.04). IndSet-Exact gives the exact chromatic number; IndSet-Removal is the heuristic upper bound.

| Graph | Family | $\lvert V\rvert$ | $\lvert E\rvert$ | Known $\chi$ | IndSet-Exact | IndSet-Removal | Exact Time | Removal Time |
|---|---|---|---|---|---|---|---|---|
| $C_8$ | Cycle | 8 | 8 | 2 | 2 | 4 | 1 ms | <1 ms |
| $C_9$ | Cycle | 9 | 9 | 3 | 3 | 3 | 7 ms | <1 ms |
| $C_{10}$ | Cycle | 10 | 10 | 2 | 2 | 4 | 24 ms | <1 ms |
| $C_{11}$ | Cycle | 11 | 11 | 3 | 3 | 3 | 86 ms | <1 ms |
| $C_{12}$ | Cycle | 12 | 12 | 2 | 2 | 4 | 168 ms | <1 ms |
| $C_{13}$ | Cycle | 13 | 13 | 3 | 3 | 3 | 786 ms | <1 ms |
| $C_{14}$ | Cycle | 14 | 14 | 2 | 2 | 4 | ≈3 s | <1 ms |
| $C_{16}$ | Cycle | 16 | 16 | 2 | 2 | 4 | ≈33 s | <1 ms |
| $K_5$ | Complete | 5 | 10 | 5 | 5 | 5 | <1 ms | <1 ms |
| $K_6$ | Complete | 6 | 15 | 6 | 6 | 6 | <1 ms | <1 ms |
| $K_7$ | Complete | 7 | 21 | 7 | 7 | 7 | <1 ms | <1 ms |
| $K_8$ | Complete | 8 | 28 | 8 | 8 | 8 | 1 ms | <1 ms |
| $K_{4,4}$ | Bipartite | 8 | 16 | 2 | 2 | 5 | 1 ms | <1 ms |
| $K_{5,5}$ | Bipartite | 10 | 25 | 2 | 2 | 6 | 9 ms | <1 ms |
| $K_{6,6}$ | Bipartite | 12 | 36 | 2 | 2 | 7 | 69 ms | <1 ms |
| $K_{7,7}$ | Bipartite | 14 | 49 | 2 | 2 | 8 | 536 ms | <1 ms |
| $GP(6,2)$ | Gen. Petersen | 12 | 18 | 3 | 3 | 4 | 135 ms | <1 ms |
| $GP(7,2)$ | Gen. Petersen | 14 | 21 | 3 | 3 | 4 | 1333 ms | <1 ms |
| $GP(8,3)$ | Gen. Petersen | 16 | 24 | 2 | 2 | 5 | ≈15 s | <1 ms |

Table 5: High chromatic number graphs: known $\chi$, IndSet-Based Exact Coloring, IndSet-Removal Coloring, and wall-clock time for each algorithm.

| Graph | $V$ | $E$ | Known $\chi$ | IndSet-Exact | IndSet-Removal | Exact Time | Removal Time |
|---|---|---|---|---|---|---|---|
| $K_9$ | 9 | 36 | 9 | 9 | 9 | 3 ms | <1 ms |
| $K_{10}$ | 10 | 45 | 10 | 10 | 10 | 9 ms | <1 ms |
| $K_{2\times5}$ | 10 | 40 | 5 | 5 | 9 | 7 ms | <1 ms |
| $K_{2\times6}$ | 12 | 60 | 6 | 6 | 11 | 47 ms | <1 ms |
| $K_{2\times7}$ | 14 | 84 | 7 | 7 | 13 | 287 ms | <1 ms |
| $K_{2\times8}$ | 16 | 112 | 8 | 8 | 15 | 1.1 s | <1 ms |
| $K_{2\times9}$ | 18 | 144 | 9 | 9 | 17 | 5.5 s | <1 ms |